D0 High-Level Design: Calorie-Based Meal Finder

Team members: Eric Braverman, Brayden Molinyawe, Christopher Farah

Diagram sources (editable in diagrams.net / draw.io or the VS Code Draw.io Integration extension): `D0_Block_Diagram.drawio`, `D0_Data_Flow.drawio`.

1. Title, Goal Statement, and Conventions

Title: Calorie-Based Meal Finder

Goal statement: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

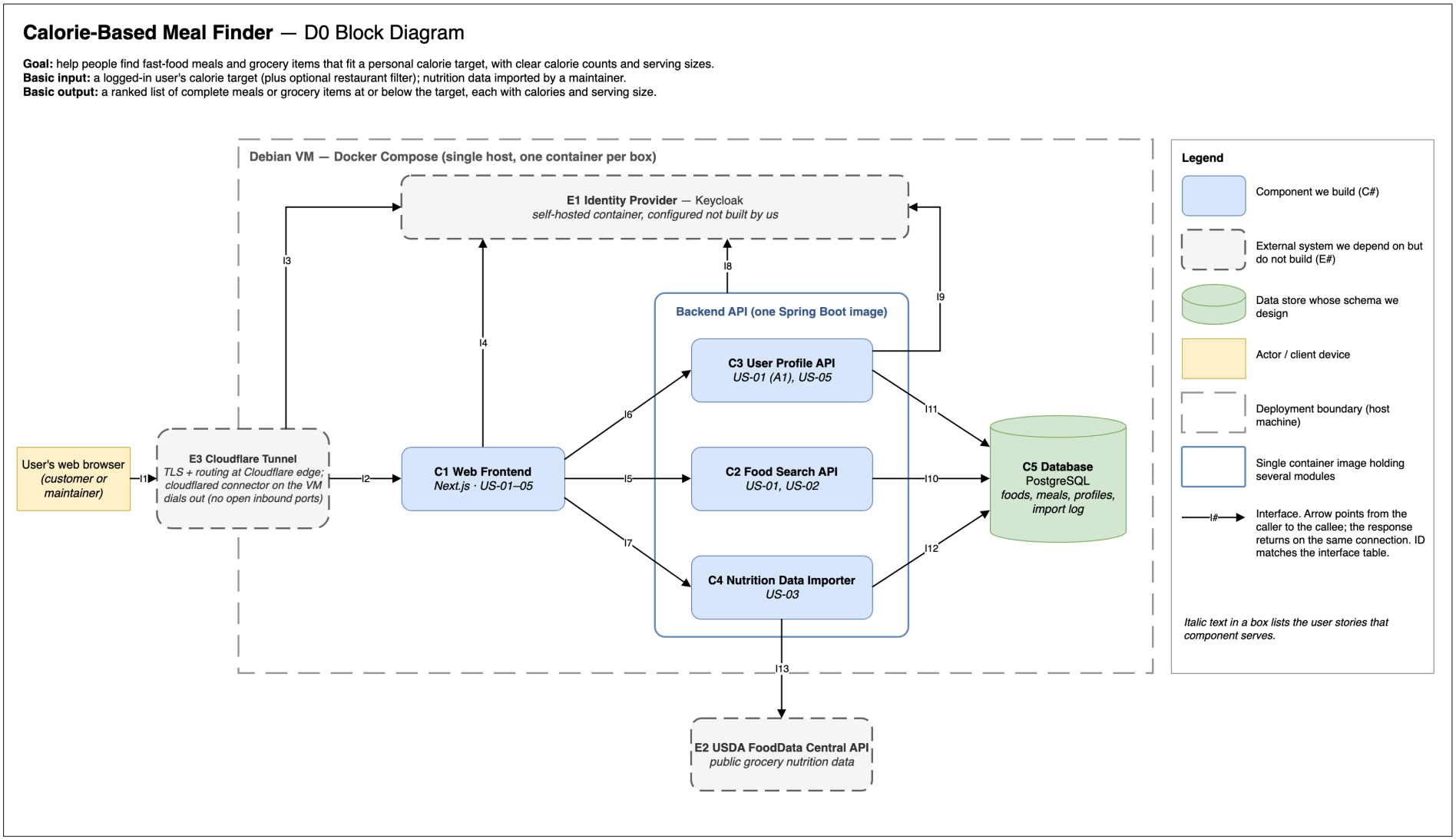
Basic input: A logged-in user's calorie target (with an optional restaurant filter), plus nutrition data that a maintainer imports from public sources.

Basic output: A ranked list of complete meals or grocery items at or below the target, each showing its calories and serving size.

Conventions: Both diagrams carry their own legend. In summary:

Symbol	Meaning
Blue rounded box (C#)	Component the team builds
Grey dashed box (E#)	External system the project depends on but does not build
Green cylinder / green open rectangle	Data store whose schema the team designs (C5 / D1)
Yellow box	Actor or client device (person, browser)
Large grey dashed boundary	Deployment boundary: the Debian VM running Docker Compose
Blue outlined group	One container image holding several modules (the Spring Boot backend)
Arrow labeled I#	Interface: points from caller to callee; the response returns on the same connection. IDs match Part 4.
Ellipse (P#)	Process in the data-flow diagram, with the component that performs it
Red dashed arrow	Exception / error flow

2. Block Diagram (D0)



Five components are built by the team (C1–C5). Three systems are external (E1 Keycloak, E2 USDA FoodData Central, E3 Cloudflare Tunnel). Keycloak and the `cloudflared` connector run as containers on our VM but are configured, not built, so they are drawn in the external style. E3 straddles the VM boundary because its connector runs on the VM while TLS and routing happen at Cloudflare's edge. C2, C3, and C4 are separate modules inside one Spring Boot container image.

3. Component Responsibility Table

ID	Component	Responsibility (one sentence)	Interfaces in	Interfaces out	Stories	Primary owner *(subject to change)*
C1	Web Frontend (Next.js)	Renders the user-facing pages, calling backend APIs on the user's behalf.	I2	I4, I5, I6, I7	US-01 – US-05 (US-04 accessibility lives here)	Brayden Molinyawe
C2	Food Search API	Returns complete meals or grocery items that fit a submitted calorie target.	I5	I8, I10	US-01, US-02	Eric Braverman
C3	User Profile API	Manages a user's saved calorie target through its full lifecycle, including account deletion.	I6	I8, I9, I11	US-01 (alt. flow A1), US-05	Christopher Farah
C4	Nutrition Data Importer	Validates incoming nutrition data before committing it to the database.	I7	I8, I12, I13	US-03	Eric Braverman
C5	Database (PostgreSQL)	Persistently stores foods, meals, calorie values, serving sizes, user profiles, and import history.	I10, I11, I12	—	US-01 – US-03, US-05	Christopher Farah
E1	Identity Provider (Keycloak) — *external*	Authenticates users and issues signed tokens.	I3, I4, I8, I9	—	all (login), US-05	Configuration: Christopher Farah
E2	USDA FoodData Central API — *external*	Supplies public grocery-item nutrition data.	I13	—	US-02, US-03	Integration: Eric Braverman
E3	Cloudflare Tunnel — *external*	Terminates public HTTPS at Cloudflare's edge, forwarding each hostname/path through an outbound-only tunnel to the matching container.	I1	I2, I3	all	Configuration: Christopher Farah

Owners are subject to change. All members work full stack and take features end to end (see `Team_Contract.md`). The primary owner reviews changes to their component and keeps its interfaces consistent with this document.

Traceability check: every story maps to at least one component (US-01→C1/C2/C3, US-02→C1/C2, US-03→C1/C4, US-04→C1, US-05→C1/C3), and every built component maps to at least one story.

4. Interface Specification Table

All internal traffic stays on the private Docker Compose network. The VM opens **no inbound ports**: the `cloudflared` connector (E3) makes an outbound connection to Cloudflare, and public traffic reaches the VM only through that tunnel. All backend calls carry `Authorization: Bearer <JWT>` issued by E1.

ID	From → To	Inputs (name: type)	Outputs (name: type)	Data format	Protocol	Error behavior (handled by)
I1	Browser → E3	HTTP request: method, path, headers, body	HTML pages, JS/CSS assets, JSON, redirects	HTML / JSON	HTTPS (TLS 1.2+) to Cloudflare's edge; certificate issued and renewed by Cloudflare	Plain HTTP → redirected to HTTPS by Cloudflare; tunnel down → Cloudflare error page 1033 (**E3**)
I2	E3 → C1	Forwarded request + `CF-Connecting-IP`, `X-Forwarded-For/Proto` headers (ingress rule: app hostname → `http://frontend:3000`)	Rendered HTML, JSON	HTML / JSON	Outbound encrypted tunnel (edge ↔ `cloudflared`), then HTTP/1.1 on the Docker network, port 3000	C1 down → 502 Bad Gateway from `cloudflared` (**E3**)
I3	E3 → E1	`/auth/*` requests: login form (username: string, password: string) entered directly on Keycloak's page	Redirect to C1 callback with code: string, state: string	HTML form / 302 redirect	OIDC Authorization Code flow; ingress rule `/auth/*` → `http://keycloak:8080` through the tunnel	Bad credentials → Keycloak re-shows its login page (**E1**); E1 down → 502 Bad Gateway from `cloudflared` (**E3**)
I4	C1 → E1	code: string, code_verifier: string, client_id: string, redirect_uri: string; later refresh_token: string	access_token: JWT (5 min), refresh_token: string, id_token: JWT, expires_in: int	`x-www-form-urlencoded` request, JSON response	OIDC token endpoint (Auth Code + PKCE), HTTP	400 `invalid_grant` or expired refresh → session cleared and user sent to login (**C1**)
I5	C1 → C2	`GET /api/v1/search` — type: `MEAL|GROCERY`, targetCalories: int (100–5000), restaurantIds: uuid[] (optional), limit: int (≤ 20)	results: SearchResult[] (id: uuid, name: string, resultType: enum, restaurant: string|null, totalCalories: int, servingSize: string, items[]), closestOption: SearchResult|null	JSON	REST over HTTP + JWT	400 `VALIDATION_ERROR` with accepted range (UC-01 E2, **C2** returns, **C1** displays); 0 matches → 200 with empty `results` + `closestOption` (UC-01 E1); 401 → re-login (**C1**); 5xx or > 2 s → "Search unavailable, try again" (**C1**)

ID	From → To	Inputs (name: type)	Outputs (name: type)	Data format	Protocol	Error behavior (handled by)
I6	C1 → C3	`GET/PUT /api/v1/profile/target` — defaultTargetCalories: int (100–5000); `DELETE /api/v1/profile`	profile: {userId: uuid, defaultTargetCalories: int|null}; 204 on delete	JSON	REST over HTTP + JWT	400 invalid target (**C3**); first GET with no profile → empty profile created (**C3**); delete fails at E1 → 502, nothing deleted (**C3** rolls back, **C1** shows retry message)
I7	C1 → C4	`POST /api/v1/imports` — restaurantId: uuid + file: CSV (≤ 5 MB) **or** source: `USDA` + query: string; `POST /api/v1/imports/{id}/confirm`	importId: uuid, status: `PENDING_CONFIRMATION|APPLIED`, added: int, removed: int, changed: {itemName, oldCalories: int, newCalories: int}[], mealsRecalculated: int	multipart/form-data request, JSON response	REST over HTTP + JWT with `maintainer` role	422 with invalidRows: {line: int, reason: string}[] and nothing saved (UC-02 E1, **C4**); 403 without `maintainer` role (**C4**); 413 file too large (**C4**)
I8	C2/C3/C4 → E1	`GET /realms/meal-finder/protocol/openid-connect/certs`	keys: JWK[] (kid, kty, alg, n, e)	JSON (JWKS)	HTTP, cached 10 min	E1 unreachable → use cached keys; no cached keys → 503; bad signature or expired token → 401 (**backend security layer**)
I9	C3 → E1	`DELETE /admin/realms/meal-finder/users/{userId}` with service-account token	204 No Content	JSON	Keycloak Admin REST API over HTTP (client-credentials token)	404 → treated as already deleted; any other error → C3 rolls back its DB delete and returns 502 (**C3**)
I10	C2 → C5	Parameterized SELECT: targetCalories: int, restaurantIds: uuid[], type: enum	Rows: meal_id: uuid, name: text, total_calories: int, serving_size: text, item rows	SQL result sets	PostgreSQL wire protocol (JDBC), TCP 5432, read-only DB role	Connection failure or query > 500 ms → 503 to C1 (**C2**)
I11	C3 → C5	SELECT / UPSERT / DELETE on user_profile keyed by Keycloak userId: uuid	Affected rows / profile row	SQL	PostgreSQL wire protocol (JDBC), TCP 5432	Constraint violation → 400; connection failure → 503 (**C3**)
I12	C4 → C5	Read current items; INSERT/UPDATE/DELETE food items, recalculated meal totals, one import_log row	Current item rows; commit result	SQL	PostgreSQL wire protocol (JDBC), single transaction	Any failure → whole transaction rolled back, data unchanged, import marked `FAILED` (**C4**)

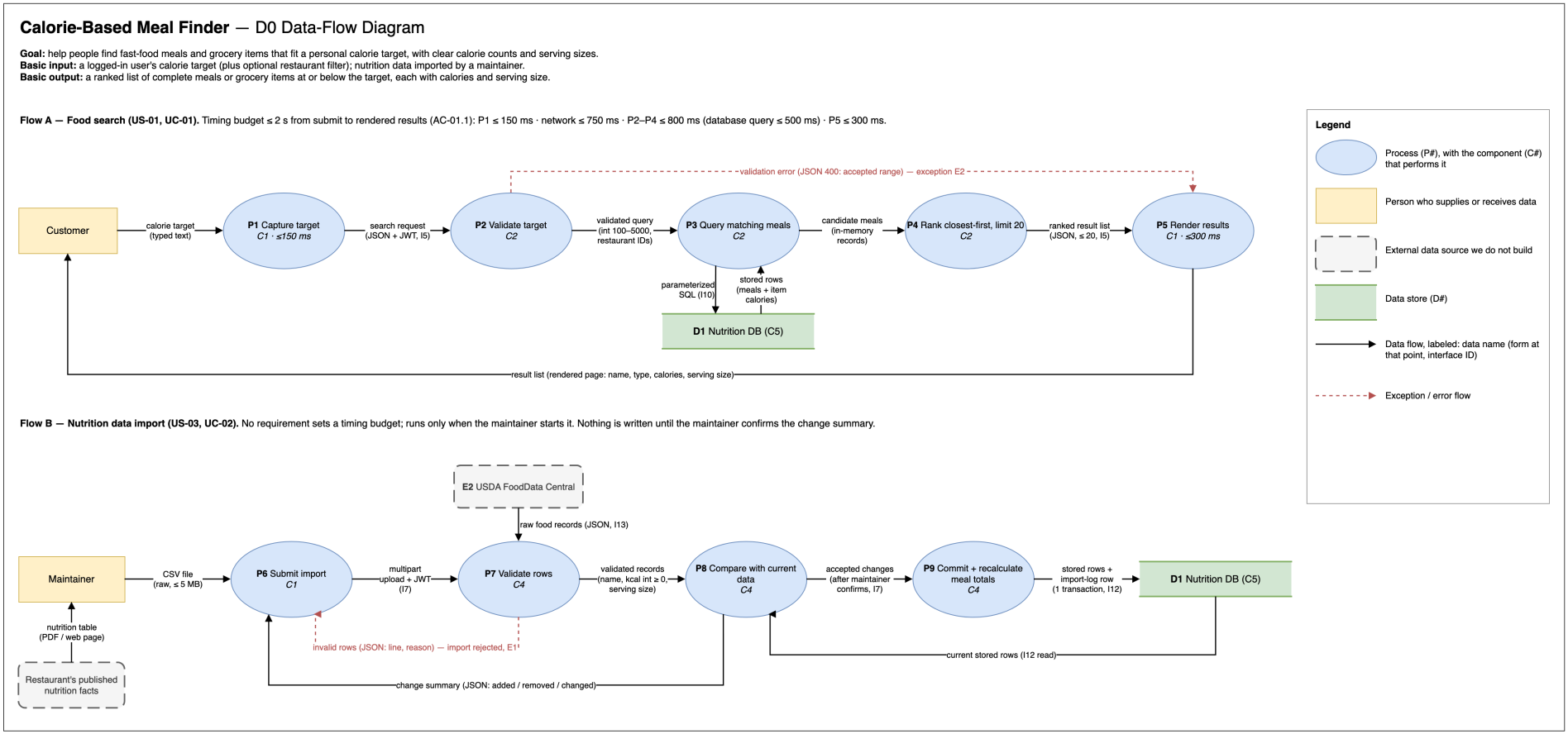
ID	From → To	Inputs (name: type)	Outputs (name: type)	Data format	Protocol	Error behavior (handled by)
I13	C4 → E2	`GET https://api.nal.usda.gov/fdc/v1/foods/search` — query: string, dataType: string[], api_key: string	foods: {fdcId: int, description: string, servingSize: number, servingSizeUnit: string, foodNutrients: {nutrientId: int, value: number}[]}[] (energy = nutrientId 1008, kcal)	JSON	REST over HTTPS	429 rate limit (1,000 req/hour per key) → back off and retry up to 3 times; 5xx → import marked `FAILED`; food with no energy value → reported as invalid row (**C4**)

Example payload — I5 (UC-01 main flow, AC-01.1):

```
GET /api/v1/search?type=MEAL&targetCalories=700&limit=20
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
```

```
{
  "results": [
    {
      "id": "6f1c2a7e-0b5d-4c1e-9a43-2d7f8e0c11aa",
      "name": "Grilled Chicken Sandwich Meal",
      "resultType": "MEAL",
      "restaurant": "Example Burger Co.",
      "totalCalories": 680,
      "servingSize": "1 meal",
      "items": [
        { "name": "Grilled Chicken Sandwich", "calories": 390, "servingSize": "1 sandwich (204 g)" },
        { "name": "Side Salad", "calories": 290, "servingSize": "1 bowl (150 g)" },
        { "name": "Diet Cola", "calories": 0, "servingSize": "16 fl oz" }
      ]
    },
    {
      "id": "1b9e44d0-77c2-4f0a-8f65-3c2a9d5e2b10",
      "name": "Turkey Wrap Meal",
      "resultType": "MEAL",
      "restaurant": "Example Deli",
      "totalCalories": 450,
      "servingSize": "1 meal",
      "items": [
        { "name": "Turkey Wrap", "calories": 450, "servingSize": "1 wrap (220 g)" }
      ]
    }
  ],
  "closestOption": null
}
```

5. Data-Flow Diagram



Flow A: food search (US-01, UC-01).

1. The customer's typed target becomes a JSON request carrying a JWT.
2. That request becomes a validated query.
3. The query returns stored rows from the database.
4. The rows are ranked into a JSON list of at most 20 results.
5. The list is rendered as a page.

The **2-second** budget from AC-01.1 is split this way:

- P1 (capture target): ≤ 150 ms
- Network: ≤ 750 ms
- P2–P4 (validate, query, rank): ≤ 800 ms, including a database query of ≤ 500 ms
- P5 (render): ≤ 300 ms

An invalid target (UC-01 E2) skips the query and returns a 400 error with the accepted range.

Flow B: nutrition data import (US-03, UC-02).

1. A raw CSV file (transcribed from a restaurant's published nutrition facts) or raw USDA JSON records come in.
2. The rows are validated into typed records.
3. The records are compared with the current stored rows, producing a change summary for the maintainer.
4. After the maintainer confirms, the changes are committed in one transaction. Affected meal totals are recalculated and an import-log row is written.

Invalid rows (UC-02 E1) reject the whole file before anything is written. No requirement sets a timing budget for imports.

6. Architecture Pattern and Justification

Chosen pattern: client-server overall, layered inside the server, with a pipeline for imports.

Pattern	Governs
Client-server	The browser (client) talks to services on one Debian VM (server) through a single HTTPS entry point (E3, Cloudflare Tunnel).
Layered	Server side in three layers: presentation (C1, reached through E3) → application/API (C2, C3, C4, packaged as one Spring Boot image) → data (C5). Each layer calls only the layer below it. Within the backend, each module is split into controller → service → data-access layers.
Pipeline	The import path in C4: validate → compare → commit. Each stage's output is the next stage's input, and the commit stage is all-or-nothing.

Justification against the five criteria:

Criterion	How the choice fits
Fit to the problem	The app is request/response: a user submits a target and gets a list. Client-server with layers matches that directly. Imports are naturally staged, which suits a pipeline.
Team skills	Christopher has shipped this exact stack (React/TypeScript, Spring Boot, PostgreSQL, Keycloak) on LexGrip. Brayden has full-stack web, API/database integration, and CI/CD experience. A layered Spring backend is the most familiar structure for all three of us.
Performance and timing	The only hard budget is 2 s per search (AC-01.1). With every component on one host, internal calls stay on the local network. Meal totals are precomputed at import time, so a search is a single indexed read.
Scalability	The expected load is a class project with tens of users. One VM is enough. If needed, more backend replicas can be added behind the tunnel later without changing the layers.

Criterion	How the choice fits
Hardware / hosting constraints	There is no device hardware. The constraint is budget (Constraints Essay, Economic): one low-cost Debian VM, free open-source components, free public data. Privacy (Constraints Essay, Security) is handled by keeping credentials in Keycloak, storing only a user ID and calorie target, and exposing no inbound ports on the VM (Cloudflare Tunnel). Cloudflare Tunnel is free, so it adds no hosting cost.

Rejected patterns:

- **Microservices:** splitting C2, C3, and C4 into independently deployed services (or adding Kubernetes) adds service discovery, more containers, and inter-service failure handling. That is extra cost and operations work that a three-person team on one VM does not need, and no requirement demands independent scaling.
- **Embedded sensor-actuator:** the system has no sensors, actuators, or real-time hardware loop.

7. Decision Log

#	Decision	Alternatives considered	Why the chosen option won
DL-01	Use **self-hosted Keycloak** as the identity provider.	(a) Hosted auth service (Auth0, Firebase Auth); (b) custom login code in Spring Boot.	Free and open source, which meets the economic constraint with no per-user pricing. It provides standard OIDC/JWT. Its Admin API supports full account deletion (US-05). Christopher has already integrated it. Custom auth was rejected as the highest security risk.
DL-02	Package C2, C3, and C4 as **modules in one Spring Boot image** (modular monolith).	(a) A separate service per module (microservices); (b) put the API logic inside the Next.js server.	One image means one build, one deployment, and one database connection pool, while each module keeps its own owner and boundary. Option (b) would mix the frontend and business logic and lose the team's Java/Spring experience.
DL-03	Deploy everything with **Docker Compose on a single Debian VM**.	(a) Kubernetes; (b) managed platforms (Vercel for the frontend + managed Postgres + hosted auth).	Compose gives reproducible containers with minimal operational overhead. Kubernetes adds complexity that the project's scale does not justify. Managed platforms spread costs across several vendors, and their free tiers limit database size and uptime.
DL-04	Use **PostgreSQL** as the single data store.	(a) MongoDB; (b) MySQL.	The data is relational: meals are made of items, totals depend on item calories, and imports must change many rows atomically. Postgres transactions and foreign keys fit this. It is free, Keycloak can use it, and the team already knows it.
DL-05	Use **Next.js** for the frontend, keeping OIDC tokens on the Next.js server.	(a) Plain React single-page app holding tokens in the browser; (b) server-rendered Spring templates.	Server-side token handling keeps JWTs out of browser storage (security constraint). Server rendering produces semantic HTML that screen readers handle well (US-04). The team has React/TypeScript experience.
DL-06	Source data from **USDA FoodData Central (grocery) and restaurant-published nutrition facts entered as CSV (fast food)**.	(a) Paid nutrition API (e.g., Nutritionix); (b) scraping restaurant websites.	Both chosen sources are free, which meets the economic constraint. The trade-off, recorded in the Constraints Essay, is manual CSV updates, which C4's validation and change summary (US-03) are designed to make safe. Scraping was rejected because it is fragile and may break site terms.
DL-07	Expose the app through **Cloudflare Tunnel** instead of running our own reverse proxy.	(a) Self-run reverse proxy (Nginx or Caddy) with ports 80/443 open and Let's Encrypt certificates; (b) exposing each container's port directly.	The tunnel is outbound-only, so the VM needs no open inbound ports or firewall rules. Cloudflare issues and renews the TLS certificate and routes by hostname/path, which removes a component the team would otherwise build and maintain. It is free, and Christopher already uses it. Option (b) would expose every service to the internet and need separate TLS setup for each.